Assignment Four, Question Two

Parts (a) - (c)

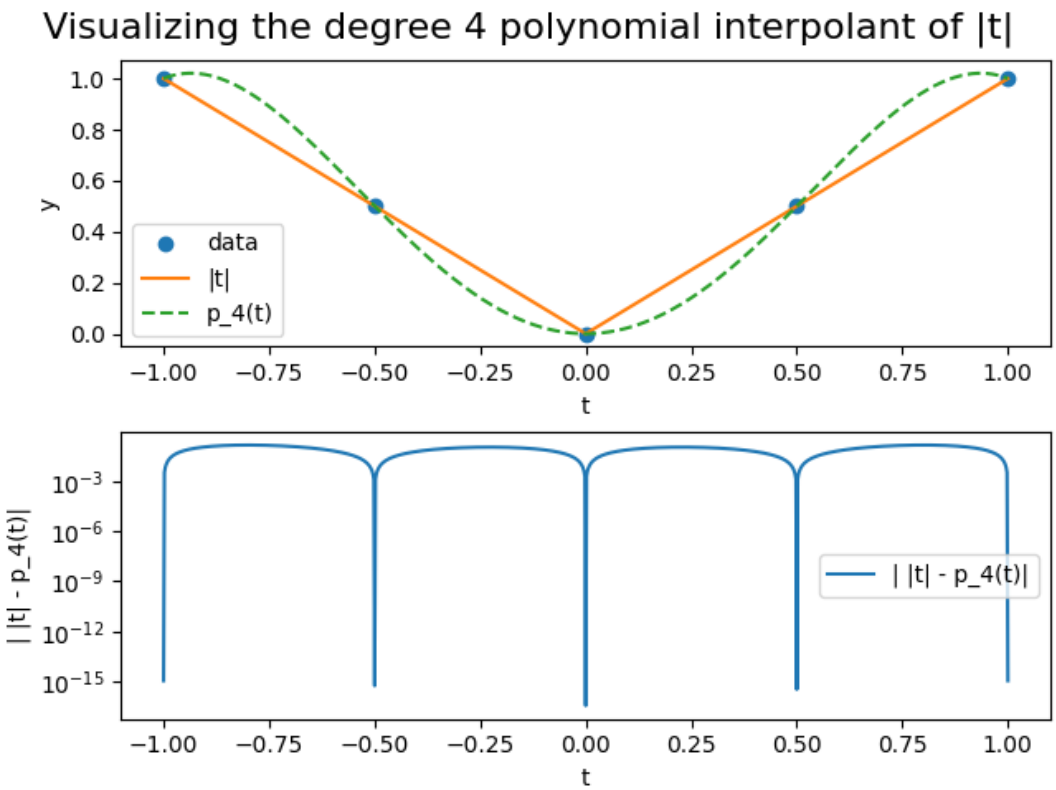
The following python program may be used to generate the required plots. It is based on the provided `csc336_a4_example.py` program.

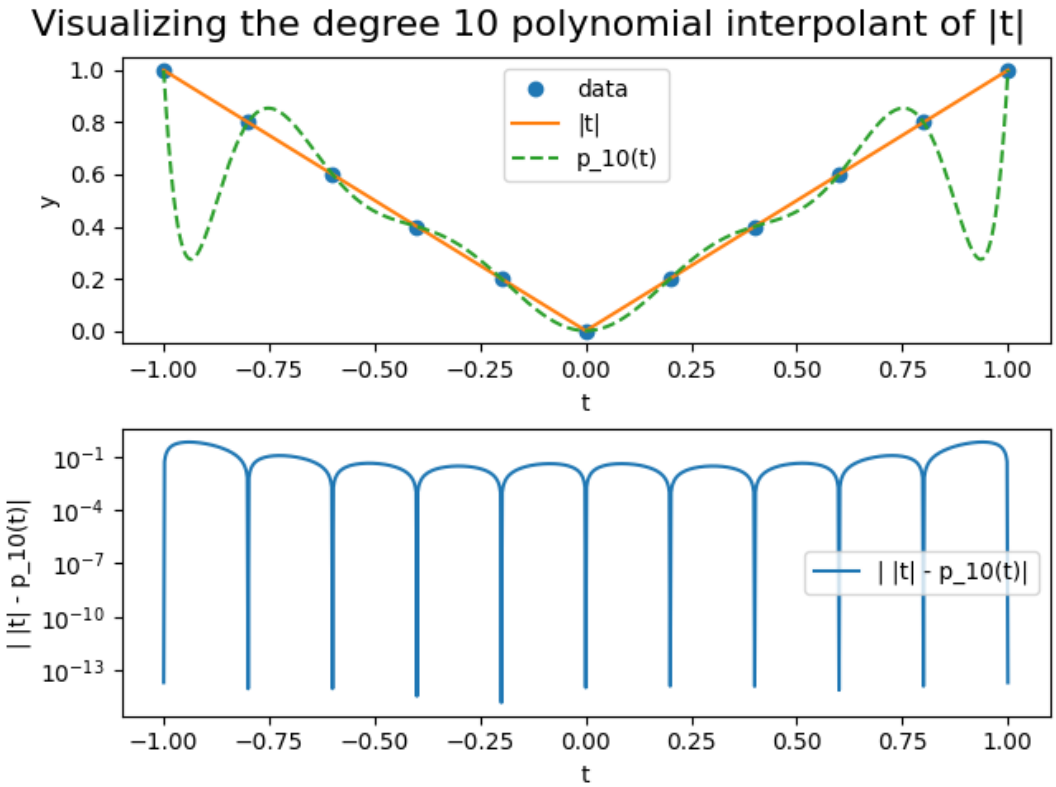
```
'''Sample solution for CSC336H1S Assignment 4, Question 2'''

import numpy as np
from numpy.polynomial.polynomial import polyfit
from numpy.polynomial.polynomial import polyval
from scipy.interpolate import CubicSpline
import matplotlib.pyplot as plt

# Parts (a) - (b)

# Using polynomial interpolation to fit f(t) = abs(t)

# points for plotting the true solution
t_plot = np.linspace(-1, 1, 1001)
y_plot = np.absolute(t_plot)

m_values = [5, 11, 21]
for m in m_values:

    # generate points to be interpolated
    t = np.linspace(-1, 1, m)
    y = np.absolute(t)

    # generate the degree m - 1 interpolant
    p_mm1 = polyfit(t, y, m - 1)

    # evaluate the interpolant at the plot points
    p_at_tplot = polyval(t_plot, p_mm1)

    # produce the plots
    fig, axs = plt.subplots(2, 1, constrained_layout=True)
    fig.suptitle(
     'Visualizing the degree {0} polynomial interpolant of |t|'.format(m - 1),
     fontsize=16)

    # plot |t| and p_{m-1}(t) vs. t
    axs[0].plot(t, y, 'o', label='data')
    axs[0].plot(t_plot, y_plot, '-', label='|t|')
    axs[0].plot(t_plot, p_at_tplot, '--',
                label='p_{0}(t)'.format(m - 1))
```

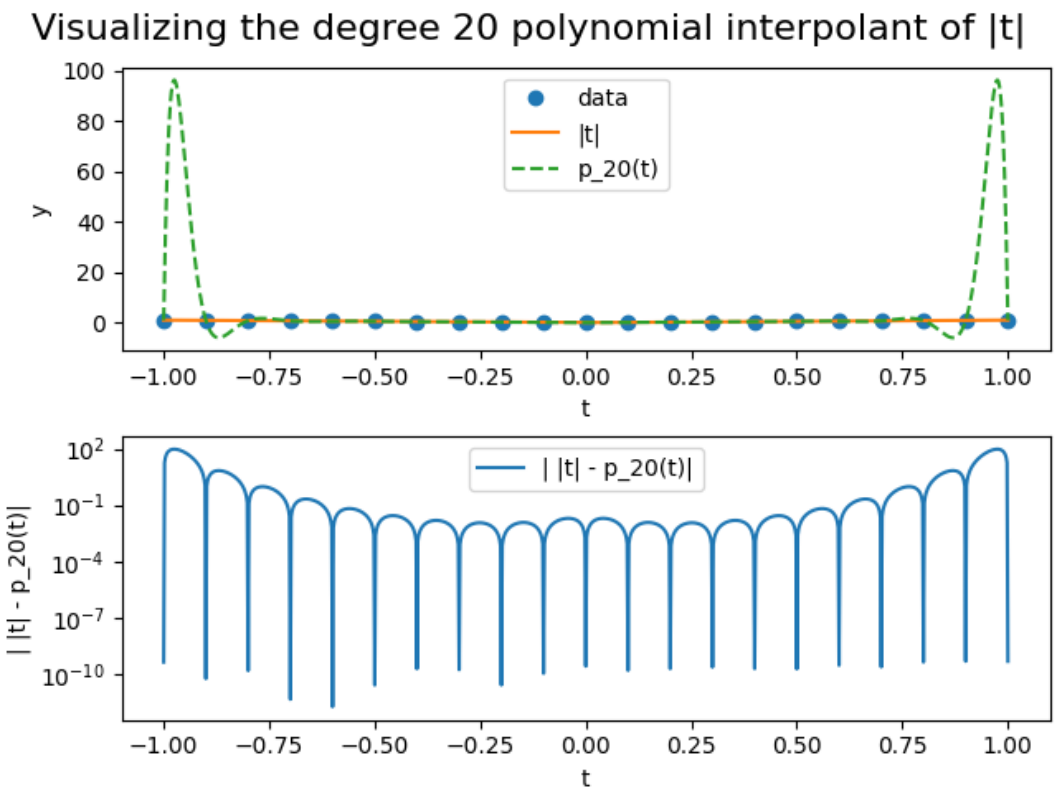
```
    axs[0].set_xlabel('t')
    axs[0].set_ylabel('y')
    axs[0].legend()

    # plot the error | |t| - p_{m-1}(t) | vs. t
    axs[1].plot(t_plot, np.abs(y_plot - p_at_tplot), '-',
                label='| |t| - p_{0}(t)|'.format(m - 1))
    axs[1].set_yscale('log')
    axs[1].set_xlabel('t')
    axs[1].set_ylabel('| |t| - p_{0}(t)|'.format(m - 1))
    axs[1].legend()


# Part (c) - repeat parts (a), (b) but use a cubic spline instead of
#    single polynomial interpolant

for m in m_values:

    # generate points to be interpolated
    t = np.linspace(-1, 1, m)
    y = np.absolute(t)

    # generate the cubic spline interpolant
    cubic_spline = CubicSpline(t, y)

    # evaluate the spline at the plot points
    cs_at_tplot = cubic_spline(t_plot)

    # produce the plots
    fig, axs = plt.subplots(2, 1, constrained_layout=True)
    fig.suptitle(
     'Visualizing the {0} point cubic spline interpolant of |t|'.format(m - 1),
     fontsize=16)

    # plot |t| and cubic_spline(t) vs. t
    axs[0].plot(t, y, 'o', label='data')
    axs[0].plot(t_plot, y_plot, '-', label='|t|')
    axs[0].plot(t_plot, cs_at_tplot, '--', label='S(t)')
    axs[0].set_xlabel('t')
    axs[0].set_ylabel('y')
    axs[0].legend()

    # plot the error | |t| - S(t) | vs. t
    axs[1].plot(t_plot, np.abs(y_plot - cs_at_tplot), '-',
                label='| |t| - S(t)|')
    axs[1].set_yscale('log')
    axs[1].set_xlabel('t')
```

```
    axs[1].set_ylabel('| |t| - S(t)|')
    axs[1].legend()

# show the plots!
plt.show()
```

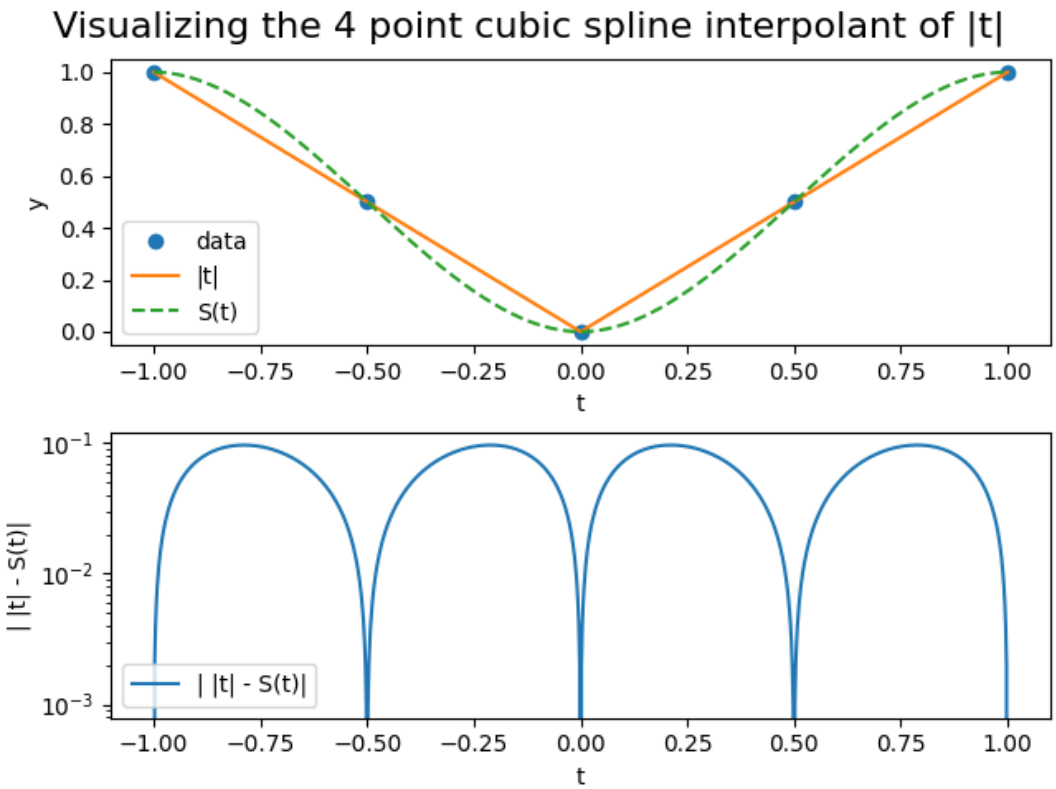
Here are the plots for parts (a) and (b) (polynomial interpolants):

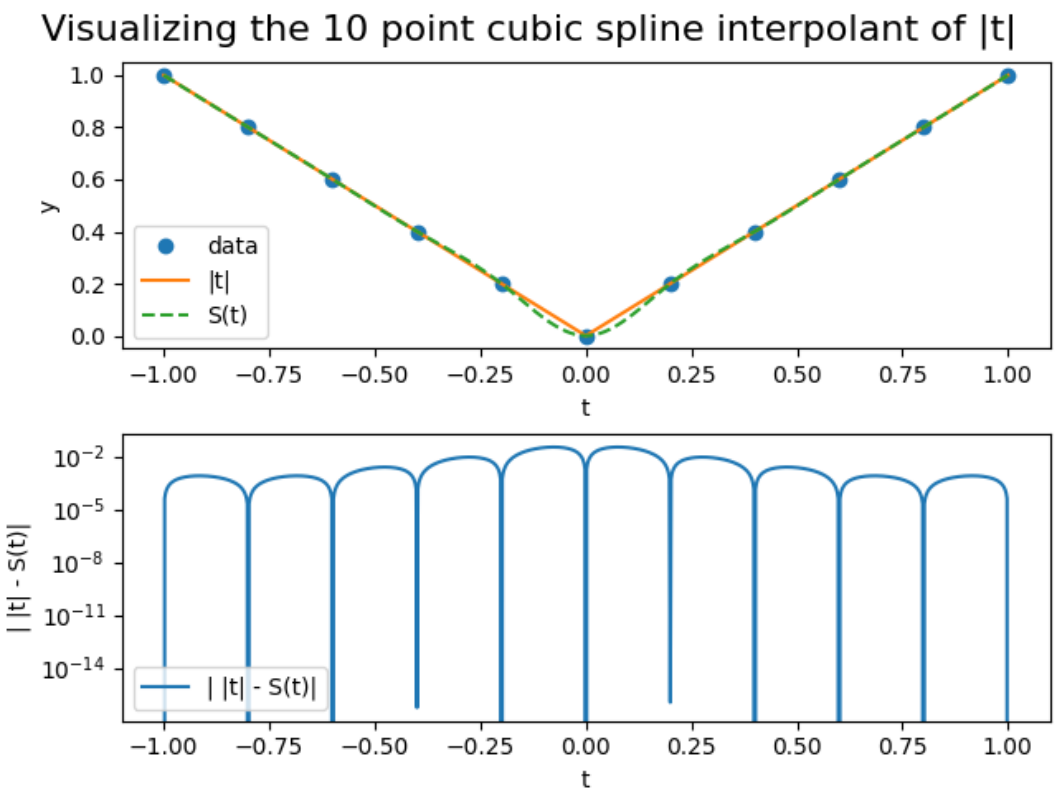


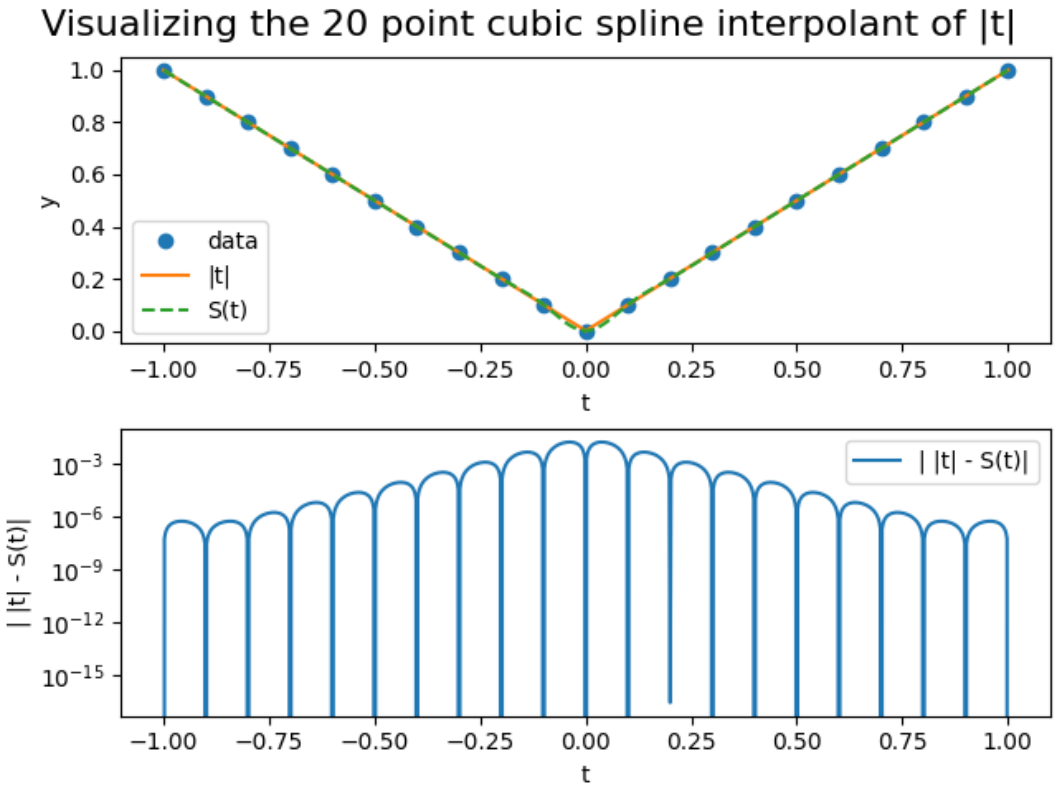

Visualizing the degree 10 polynomial interpolant of |t|
data
|t|
p_10(t)
y
t
| |t| - p_10(t)|
t


Visualizing the degree 20 polynomial interpolant of |t|
data
|t|
p_20(t)
y
t
| |t| - p_20(t)|
t

Here are the plots for parts (c) (cubic splines):







Part (d) - Comment on your observations:

We observe the following:

- For polynomial interpolation on equally spaced points, the approximation error increases as the number of interpolation points is increased. This is most evident near the the endpoints of the interval.
- For cubic spline interpolation, we observe that the approximation error does decrease as more points are included. Note that the error is largest near the "cusp" point at $t = 0$ where $f(t)$ is not differentiable.

The reason for the poor results with the single polynomial interpolant is that $f(t)$ is not differentiable throughout the interval – it is not defined at $t = 0$. This leads to the observed large errors at the endpoints.

We don't have the same issue with the cubic spline as the error bound depends on $f^4(t)$ and the bound on it will not change with increasing m.